

RFP Intelligence & Monitoring Platform

Full Architecture, AI Agent Design, Database, API, Notification & Antigravity Development Specification

Objective: Build a production-ready platform supporting both direct individual RFP URLs and recurring RFP source/listing websites. The system extracts structured RFP information, stores it, detects duplicates and changes, provides search/filtering, manages users and role-based permissions, and emails eligible users when new RFPs are discovered.

1. Product Scope

The platform has two ingestion modes. **Mode A — Individual RFP URL:** a user/admin submits a direct RFP page or document URL and the AI pipeline extracts the RFP immediately. **Mode B — RFP Source Website:** an administrator registers an RFP listing/source URL; a scheduled crawler checks it every day, discovers candidate RFPs, identifies new records, extracts them, validates them, stores them and triggers notifications.

Frontend and backend are separate applications communicating through versioned REST APIs. Long-running crawling, AI extraction and email delivery run as background jobs.

2. Two Ingestion Modes

Mode	Input	Purpose	Recurring?
A — Individual RFP	Direct RFP page/PDF URL	Extract one specific RFP and save it	Optional
B — Website Monitoring	RFP listing/source URL	Find new RFPs from a source every day	Yes

3. High-Level Architecture

Frontend (Next.js/React) → REST API (FastAPI) → application services → PostgreSQL. The backend scheduler creates jobs in Redis. Crawl workers fetch pages/documents. The AI Agent operates through controlled tools for classification, extraction and interpretation. Validated results are persisted. Notification matching creates email jobs processed by an email worker.

Frontend	API / Business	Async / AI	Storage / External
Auth Dashboard RFP List/Details Websites Users/Roles Notifications	FastAPI Auth/RBAC RFP service Website service Notification service Audit	Scheduler Redis Queue Crawl Worker AI Agent Worker Email Worker	PostgreSQL Object Storage RFP Websites Email Provider

4. AI Agent — Exact Purpose

The AI Agent is the intelligence layer. It should not own scheduling, database writes, authentication, email delivery or unrestricted browser control. It receives fetched content and uses explicit tools to decide what the content means and to produce structured, auditable results.

Capability	Purpose	Example
RFP Classification	Decide whether a page/document is an RFP/tender/procurement opportunity; supplier registration=false	
Field Extraction	Map unstructured content to the standard RFP schema.	Find deadline despite different wording
Document Understanding	Interpret PDF/DOCX and identify relevant attachments.	Identify technical specification
Ambiguity Detection	Flag uncertain/conflicting facts instead of inventing data.	Web deadline differs from PDF
Change Interpretation	Distinguish meaningful changes from cosmetic HTML changes.	Deadline changed; tracking parameter ignored

5. Agent Tool Contract

Expose narrow tools/functions such as `fetch_page(url)`, `render_page(url)`, `extract_links(content)`, `download_document(url)`, `parse_pdf(file)`, `parse_docx(file)`, `classify_candidate(content)`, `extract_rfp(content)`, `validate_rfp(data)`, and `compare_rfp_versions(old,new)`. The backend controls tool execution, permissions, timeouts and allowed domains.

Never allow the agent direct arbitrary SQL, unrestricted filesystem access, arbitrary email sending or unvalidated database mutation.

6. Mode A — Individual RFP URL Workflow

The user/admin submits a direct RFP URL. The backend validates the URL, fetches it using HTTP or Playwright, discovers linked documents, extracts readable text and sends the content to the AI Agent.

```
URL submitted
→ URL validation
→ Fetch page
→ Render if required
→ Find attachments
→ Parse HTML/PDF/DOCX
→ AI classification
→ AI extraction
→ Schema validation
→ Duplicate check
→ Save RFP
→ Optional notification
```

The result should clearly state NEW, EXISTING, UPDATED or FAILED. If the URL already represents a stored RFP, update/compare it rather than creating a duplicate.

7. Mode B — Daily Website Monitoring Workflow

An administrator registers a source/listing URL. The scheduler checks active sources every day. A crawl worker fetches the source, handles pagination/browser rendering where needed, discovers candidate RFP links/documents and passes candidates into the AI pipeline.

```
Scheduler
→ Active websites
→ Crawl jobs
→ Source fetch
→ Candidate discovery
→ Candidate classification
→ New/existing decision
→ Extraction
→ Validation
→ Persistence
→ Notification matching
```

Use a generic crawler. Do not create a separate scraper application for every website. Allow optional site-specific configuration only when generic discovery cannot reliably handle a source. Provide a mandatory Test Crawl before activation.

8. New RFP Detection & Identity

Use identity signals in priority order: external RFP/reference number; canonical source URL; source website + normalized title + deadline; content fingerprint/hash. Store canonical URLs and normalized values and use database uniqueness constraints where safe.

A repeated discovery updates the existing RFP instead of inserting a duplicate. Existing records are compared with their previous version for meaningful changes.

9. RFP Data Model

Core fields: id, website_id/source_id, external_rfp_id, title, reference_number, organization, description, published_date, submission_deadline, estimated_budget, currency, location, eligibility, requirements, submission_method, contact_person, contact_email, contact_phone, source_url, status, content_hash, first_seen_at, last_seen_at, created_at, updated_at.

Suggested statuses: NEW, OPEN, CLOSED, CANCELLED, UPDATED, EXTRACTION_FAILED, NEEDS_REVIEW.

10. AI Extraction Schema

```
{
  "title": "",
  "reference_number": "",
  "organization": "",
  "description": "",
  "published_date": null,
```

```
"submission_deadline": null,  
"estimated_budget": null,  
"currency": null,  
"location": null,  
"category": null,  
"eligibility": [],  
"requirements": [],  
"submission_method": null,  
"contact_person": null,  
"contact_email": null,  
"contact_phone": null,  
"documents": [],  
"source_url": "",  
"confidence": 0.0,  
"validation_flags": []  
}
```

Never hallucinate missing values. Use null for unknown values. Normalize dates to ISO format. Preserve source URLs and source evidence where possible.

11. Validation Pipeline

AI output
→ JSON schema validation
→ Field/business validation
→ Source consistency checks
→ Duplicate check
→ Confidence threshold
→ Persist OR NEEDS_REVIEW

Examples: deadline must be a valid date; published date should not normally be after the deadline; email must be syntactically valid; conflicting website/PDF values should be flagged.

12. Change Detection & Versioning

For monitored RFPs, retain previous versions or a change log. Detect meaningful changes to deadline, budget, eligibility, requirements, submission method, status, title and documents. Ignore cosmetic HTML changes, tracking parameters and irrelevant layout changes.

Field	Old	New	Action
Deadline	2026-09-20	2026-10-05	Record meaningful change
Budget	\$100k	\$120k	Record meaningful change
Tracking parameter	?utm=x	?utm=y	Ignore

13. Database / ERD-Level Design

Table	Purpose / key fields
users	id, name, email, password_hash, role_id, status, verification and timestamps
roles	id, name, description
permissions	id, key, description
role_permissions	role_id, permission_id
websites	id, name, base_url, start_url, status, crawl_frequency, config, timestamps
crawl_runs	id, website_id, status, metrics, timestamps, error
rfps	id, website_id, identity fields, normalized RFP fields, status, hashes, timestamps
rfp_documents	id, rfp_id, name, type, source_url, storage_url, hash
rfp_versions	id, rfp_id, version_no, snapshot, content_hash, created_at
rfp_changes	id, rfp_id, versions, field, old_value, new_value, detected_at
notification_preferences	id, user_id, email_enabled, new_rfp_enabled, filters
notifications	id, user_id, rfp_id, type, status, created_at, sent_at
email_logs	id, notification_id, provider, status, attempts, error, timestamps
audit_logs	id, actor_id, action, entity_type, entity_id, metadata, created_at

14. User Management & RBAC

Implement Users → Roles → Permissions. Suggested roles: Super Admin, Admin, RFP Manager, Standard User. Permissions should include users.read/create/update/delete; roles.read/create/update/delete; websites.read/create/update/delete/crawl; rfps.read/create/update/delete; notifications.read/manage; audit_logs.read.

Backend authorization is mandatory. Frontend checks are only for UX and must never be the security boundary.

15. Notification Eligibility Engine

When a genuinely new RFP is persisted, evaluate user eligibility. At minimum: email_enabled=true and new_rfp_enabled=true. Design for future filters including organization/company, category, location, keywords and RFP type.

```
New RFP
→ Query eligible users
→ Evaluate filters
→ Create one notification per matching user
→ Queue email
→ Send
→ Record status
```

Prevent duplicates using a uniqueness rule such as user_id + rfp_id + notification_type.

16. Email Notification

Email contains RFP title, organization, reference number, publication date, deadline, short description, source/details link and relevant documents where appropriate. Use an asynchronous worker, retries and delivery logging. Notification states: PENDING, QUEUED, SENT, FAILED.

17. Frontend Information Architecture

Authentication: Login, Forgot Password, Reset Password.

Dashboard: total RFPs, new today, open, closing soon, active sources, crawl failures, notification metrics.

RFPs: list, details, search, organization/company, publication/deadline ranges, status, website/source, category, pagination, sorting.

Websites: list, create, edit, activate/deactivate, Test Crawl, details, crawl history.

Users: list, create, edit, status, role assignment, notification preferences.

Roles: role CRUD and permission assignment.

Notifications: history, delivery status and preferences.

18. Backend API

```
/api/v1/auth
/api/v1/users
/api/v1/roles
/api/v1/permissions
/api/v1/websites
/api/v1/websites/{id}/crawl
/api/v1/crawl-runs
/api/v1/rfps
/api/v1/rfps/{id}
/api/v1/rfps/{id}/documents
/api/v1/rfps/import-url
/api/v1/notifications
/api/v1/notification-preferences
/api/v1/audit-logs
```

19. Backend Module Structure

```
app/
  api/
  models/
  schemas/
  repositories/
  services/
    auth/
    users/
    rbac/
    websites/
    crawler/
    extraction/
    validation/
    duplicate_detection/
    change_detection/
    notifications/
    email/
  agents/
    rfp_agent/
    tools/
    prompts/
    schemas/
  workers/
    crawl_worker
    extraction_worker
```

```
email_worker
core/
tests/
```

20. Recommended Technology Stack

Layer	Recommended
Frontend	Next.js / React, TypeScript, React Query
Backend	Python, FastAPI, SQLAlchemy
Database	PostgreSQL
Queue	Redis + Celery/RQ or equivalent
Browser	Playwright
Parsing	BeautifulSoup/lxml + PDF/DOCX parsers
AI	LLM with structured outputs and tool calling
Documents	S3-compatible object storage
Email	SES / Resend / SendGrid or equivalent
Deployment	Docker; separate frontend, API, workers, DB and Redis

21. Scheduling & Worker Architecture

Use a scheduler to enqueue due website crawls. API servers must remain responsive. Route crawl, AI extraction and email workloads to background workers.

```
Scheduler
→ Redis
→ Crawl Worker
→ Extraction Job
→ AI Agent Worker
→ Validation/Persistence
→ Notification Job
→ Email Queue
→ Email Worker
```

A failed source must not stop the daily process. Use retry/backoff for transient failures and controlled retries for AI failures.

22. Security

Use secure password hashing, authentication sessions/tokens, backend RBAC, CORS restrictions, input validation, rate limiting where appropriate, secret management, audit logging and least-privilege service credentials. Restrict agent network access to approved/validated domains where practical. Protect stored documents and user information.

23. Observability

Every crawl, extraction, notification and email should have traceable IDs. Log status, duration, counts, source identifiers and errors. Provide operational views for failed crawls, extraction failures, RFPs needing review and failed emails.

24. Detailed Development Plan for Antigravity

Phase	Scope	Acceptance checkpoint
1	Separate frontend/backend repositories, Docker, PostgreSQL, environment files, API contracts, migrations work	Health check API works
2	Authentication, users, roles, permissions, authorization middleware	RBAC tests pass
3	Website/source management, crawl history, Test Crawl	Admin can add/activate/test source
4	Crawler, Playwright, Redis queue, scheduler, retries	Async crawl completes and is logged
5	AI Agent, tools, prompts, schema, PDF/DOCX understanding	Individual RFP URL produces validated JSON
6	RFP persistence, duplicate detection, list/details/search/filter	New/repeated URLs behave correctly
7	Daily monitoring, versioning, change detection	Updates are detected without duplicates
8	Eligibility engine, notifications, email queue/provider, logs	Eligible users get one email
9	Dashboard, audit logs, operational monitoring	Admin can see health and metrics
10	Integration tests, security, performance, deployment	End-to-end flow passes reliably

25. Antigravity Implementation Rules

1. Implement incrementally; do not generate the whole system in one uncontrolled step.
2. Inspect the existing repository before each phase and preserve working code.
3. Keep frontend and backend separate with an explicit API contract.
4. Use migrations for database schema changes.
5. Keep long-running tasks out of HTTP request handlers.
6. Keep AI prompts, schemas and tools versioned.
7. Do not allow AI to directly mutate the database without backend validation.
8. Add automated tests at every phase and fix failures before moving forward.
9. Use realistic sample RFP pages and PDF documents for testing.
10. Implement logging and observability from the beginning.

11. Do not create a custom scraper module for every website; use a generic crawler plus data-driven site configuration when required.

26. End-to-End Acceptance Scenario

1) Admin adds a source website and runs Test Crawl. 2) System discovers RFP candidates. 3) Admin activates daily monitoring. 4) Scheduler runs next day. 5) A new RFP is discovered. 6) AI extracts title, organization, reference number, dates, requirements, eligibility and documents. 7) Validation passes. 8) RFP is inserted once. 9) Notification engine evaluates users. 10) Eligible users receive one email. 11) Next crawl does not create a duplicate. 12) If deadline changes, a version/change is recorded. 13) Admin can search by company, dates, deadline and other filters. 14) Crawl, extraction and email failures are visible.

27. Final Product Principle

The product is an RFP intelligence platform, not simply a scraper. The crawler obtains source material; the AI Agent understands and structures it; validation protects data quality; PostgreSQL is the system of record; scheduler/workers provide reliable recurring execution; RBAC controls access; and the notification engine delivers new opportunities to the right users.